# VS Code Codespaces

The SWAT+ repository ships a `.devcontainer/` folder that lets GitHub Codespaces spin up a ready-to-use Visual Studio Code instance in the browser. No local Fortran toolchain needed.

When a Codespaces instance starts, it:

- loads your fork into a remote working folder
- installs the Intel `ifx` and `gfortran` compilers
- configures VS Code to use the CMake build system
- installs the VS Code extensions needed to debug Fortran
- copies the `refdata/Ames_sub1` dataset into `/workdata/Ames_sub1`
- creates an empty `/workdata/my_data` folder for your own inputs

Screenshots referenced below live in the SWAT+ source repo's `doc/` folder.

## Cost

GitHub gives every account 120 free CPU hours per month across all Codespaces instances. A 2-CPU instance running for 60 hours uses 120 hours of quota. Beyond that, you pay per active hour. A small per-hour storage fee applies whether the instance is running or stopped.

Instance states:

- **Active.** The instance is in use (interactive or compiling). After 30 minutes of inactivity it goes to **Inactive**. The timeout is configurable.
- **Inactive.** Not running. Storage fee only.
- **Shutdown.** You stopped it manually. Same as Inactive billing-wise.
- **Deleted.** No further charges. A new instance has to be created from scratch to come back.

## Starting an instance

1. Sign in to GitHub and go to your `swatplus` fork.
2. Open the hamburger menu (top left) and click **Codespaces**.
3. Click **New codespace** (top right).
4. Pick the repository, the branch, and the CPU count. Two CPUs is enough for SWAT+. Click **Create codespace**.

The instance takes a couple of minutes to come up. You should see an install progress bar in the lower right. When it disappears, a drop-down at the top of the editor asks which Fortran compiler to use. Pick one.

## Building

Click the CMake icon in the left sidebar (the triangle-with-wrench) to open the CMake pane.

1. Under **Project Status > Configure**, check the Fortran compiler shown. To change it, hover and click the pencil icon, then pick another.
2. Right-click `CMakeLists.txt` in the CMake pane and select **Clean and Reconfigure**. Do this every time you switch compilers, and any time you add or remove a Fortran source file.
3. Right-click `CMakeLists.txt` again and select **Build All Projects**. The Output panel at the bottom should end with:

```
[build] Build finished with exit code 0
```

## Running and debugging

Click the debug icon in the left sidebar (the triangle with a bug). Pick `Ames_sub1` from the drop-down and click the green triangle.

After a fresh build, VS Code may ask which executable to run. Any of the debug-build options works. A release build is not debuggable.

Watch model output in the **Terminal** tab at the bottom. Set breakpoints, conditional breakpoints, or just let it run. On a runtime exception, the debugger stops at the failure point and you can inspect variable values.

## Using your own dataset

The `/workdata/my_data` folder starts empty. To populate it:

1. In the VS Code Explorer, right-click `/workdata/my_data` and select **Upload**.
2. Allow the cookie prompt if your browser shows one (GitHub needs it to open your local file picker).
3. Select all the files in your input dataset and upload.
4. Switch the debug drop-down to `my_data` and click the green triangle.

To add a third dataset, create a new folder under `/workdata/`, then edit `.vscode/launch.json`. The default file already has blocks for `Ames_sub1` and `my_data`. Copy one block, change `name` and `cwd`, and add a comma between blocks.

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Ames_sub1",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/Ames_sub1",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        },
        {
            "name": "walnut_creek",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
```

```
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/walnut_creek",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

## Stopping the instance

To stop billing for active usage, click **Codespaces** in the bottom-left status bar and select **Stop Current Codespace**. It takes a minute or two.

## Deleting the instance

After stopping, go back to the browser tab where you created the instance (or open the Codespaces panel from your GitHub repository). Find the instance at the bottom, click the ellipsis on its row, and select **Delete**. Storage charges stop after deletion. A new instance has to be created from scratch the next time you want to work.